GovBiz

AI 기반 정부지원사업 탐색·신청 관리 플랫폼

여러 기관에 분산된 공고를 모아 기업에 맞는 사업을 찾고,
신청 준비부터 진행 관리까지 하나의 서비스에서 연결합니다.

PRODUCT

웹·모바일

QUALITY

평가 / 예산 / 실행 추적

OPERATION

K8s / GitOps / 자동 확장

어떤 문제를 해결하는가

공고를 찾는 과정과 신청을 준비하는 과정을 하나의 흐름으로 연결합니다.

01 탐색의 어려움

기관마다 흩어진 공고
서로 다른 검색·필터 방식
우리 기업과의 적합성 판단

02 근거 확인의 부담

지원 대상·신청 조건 확인
긴 원문과 첨부 서식 검토
추천 이유의 출처 확인

03 준비 과정의 단절

답변·신청 서류 준비
마감·진행 단계 관리
협업 파트너 탐색

공고 탐색 → 공식 근거 확인 → 신청 준비 → 진행·협업 관리

3차 기반과 4차 확장 범위

유지한 기술과 바뀐 구성을 같은 역할끼리 비교합니다.

영역	3차 프로젝트 기반	4차 프로젝트 확장
서비스	공고 검색·근거 답변·신청 준비 중복 검토·협업 파트너 탐색	**Supervisor로 모든 에이전트 기능 통합**
클라이언트	웹 중심 사용자 경험	**Expo 모바일 + 공통 계약·로직 공유**
품질 운영	개별 AI 기능·평가 기록	**평가 접수·예산·검토·실행 추적**
실행 환경	EC2 + Docker Compose	**Kubernetes(EKS) + EC2 노드**
이미지 저장	Amazon ECR	**Amazon ECR (동일)**
배포 반영	SSM으로 EC2의 Compose 갱신	**Argo CD로 Kubernetes 배포 동기화**
자동 확장	자동 확장 구성 기록 없음	**HPA(Pod) + Cluster Autoscaler(EC2)**

서비스 책임을 네 영역으로 분리

사용자 업무, 공식 데이터, AI 추론, 운영 평가가 각자의 계약을 소유합니다.

Core Service

사용자 업무의 진입점

인증·권한, 관심 공고, 신청·협업
AI·Catalog 호출과 결과 검증

Kotlin · Spring Boot · MyBatis

Catalog Service

공식 공고의 수집과 발행

제공처별 수집·정규화
검색 색인 준비 후 snapshot 발행

Kotlin · Spring Boot

AI Service

검색·생성·분류 실행

OpenAI 기반 추론
근거 답변·Supervisor·문서 지원

Python · FastAPI · OpenAI

Ops Service

평가와 운영 정책

평가 명세·예산·검토 상태
Prefect 실행·결과·추적 연결

Python · Django · MySQL

웹과 모바일이 같은 업무 계약을 사용

화면은 각 플랫폼에 맞추고, 공통 DTO·API 계약과 업무 로직은 Shared에 모읍니다.

Web

React 기반 서비스 화면
관리자 LLMOps 화면
Vercel을 통한 웹 제공

Shared

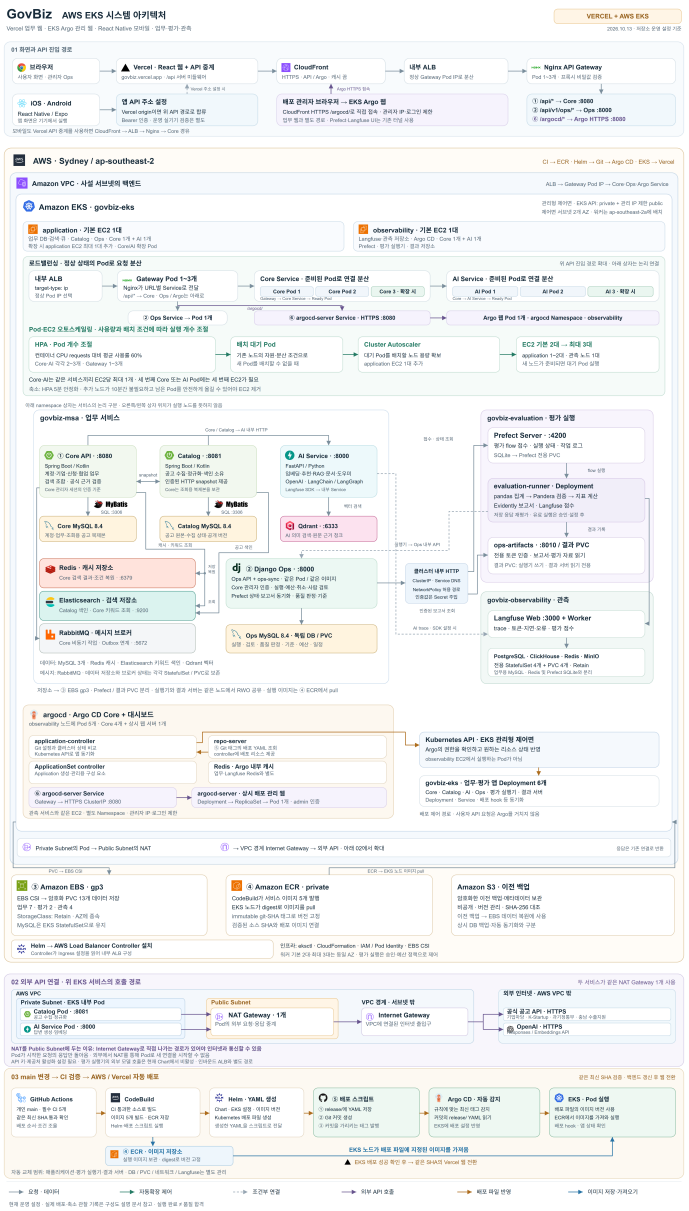
공통 DTO·API 계약
검색·필터 등 공통 업무 로직
웹·모바일에서 함께 사용

Mobile

React Native · Expo
관리자 Gov 에이전트 포함
Firebase App Distribution
팀 테스트 빌드 배포

브라우저와 앱의 사용 경로를 넓히면서, 계약 변경에 따른 차이를 함께 관리합니다.

Firebase App Distribution은 모바일 테스트 배포에 사용합니다.



Vercel · AWS EKS

운영 전체 구성도

웹·모바일의 API 진입 경로

업무·평가·관측 서비스와 데이터 저장소

로드밸런싱·자동 확장·GitOps 배포

운영 구성도 ① 진입·분산·자동 확장

GovBiz AWS EKS 시스템 아키텍처

Vercel 업무 웹 · EKS Argo 관리 웹 · React Native 모바일 · 업무·평가·관측

VERCEL + AWS EKS

2026.10.13 · 저장소 운영 설정 기준

01 화면과 API 진입 경로

- **브라우저** — 사용자 화면 · 관리자 Ops
- **Vercel · React 웹 + API 중계** — govbiz.vercel.app · /api 서버 미들웨어
- **CloudFront** — HTTPS · API / Argo · 캐시 끔
- **내부 ALB** — 정상 Gateway Pod IP로 분산
- **Nginx API Gateway** — Pod 1~3개 · 프록시 비밀값 검증
- **iOS · Android** — React Native / Expo · 앱 화면은 기기에서 실행
- **앱 API 주소 설정** — Vercel origin이면 위 API 경로로 합류 · Bearer 인증 · 운영 실기기 검증은 별도
- **배포 관리자 브라우저 → EKS Argo 웹** — CloudFront HTTPS /argocd/로 직접 접속 · 관리자 IP·로그인 제한 · 업무 웹과 별도 경로 · Prefect·Langfuse UI는 기존 터널 사용
- ① /api/* → Core :8080
- ② /api/v1/ops/* → Ops :8000
- ⑥ /argocd/* → Argo HTTPS :8080

Vercel 주소 설정 시 · Argo HTTPS 접속

모바일도 Vercel API 중계를 사용하면 CloudFront → ALB → Nginx → Core 경유

AWS · Sydney / ap-southeast-2

CI → ECR · Helm → Git → Argo CD · EKS → Vercel

Amazon VPC · 사설 서브넷의 백엔드

ALB → Gateway Pod IP → Core·Ops·Argo Service

Amazon EKS · govbiz-eks

관리형 제어면 · EKS API: private + 관리 IP 제한 public
제어면 서브넷 2개 AZ · 워커는 ap-southeast-2a에 배치

- **application · 기본 EC2 1대** — 업무 DB·검색·큐 · Catalog · Ops · Core 1개 + AI 1개 · 확장 시 application EC2 최대 1대 추가 · Core/AI 확장 Pod
- **observability · 기본 EC2 1대** — Langfuse·관측 저장소 · Argo CD · Core 1개 + AI 1개 · Prefect · 평가 실행기 · 결과 저장소

로드밸런싱 · 정상 상태의 Pod로 요청 분산

위 API 진입 경로 확대 · 아래 상자는 논리 연결

- **내부 ALB** — target-type: ip · 정상 Pod IP 선택
- **Gateway Pod 1~3개** — Nginx가 URL별 Service로 전달 · /api/* → Core · Ops / Argo는 아래로
- **Core Service · 준비된 Pod로 연결 분산** — Core Pod 1 · Core Pod 2 · Core 3 · 확장 시 (Gateway → Core Service → Ready Pod)
- **AI Service · 준비된 Pod로 연결 분산** — AI Pod 1 · AI Pod 2 · AI 3 · 확장 시 (Core → AI Service → Ready Pod)
- ② Ops Service → Pod 1개
- /argocd/
- ⑥ argocd-server Service · HTTPS :8080
- Argo 웹 Pod 1개 · argocd Namespace · observability

Pod·EC2 오토스케일링 · 사용량과 배치 조건에 따라 실행 개수 조절

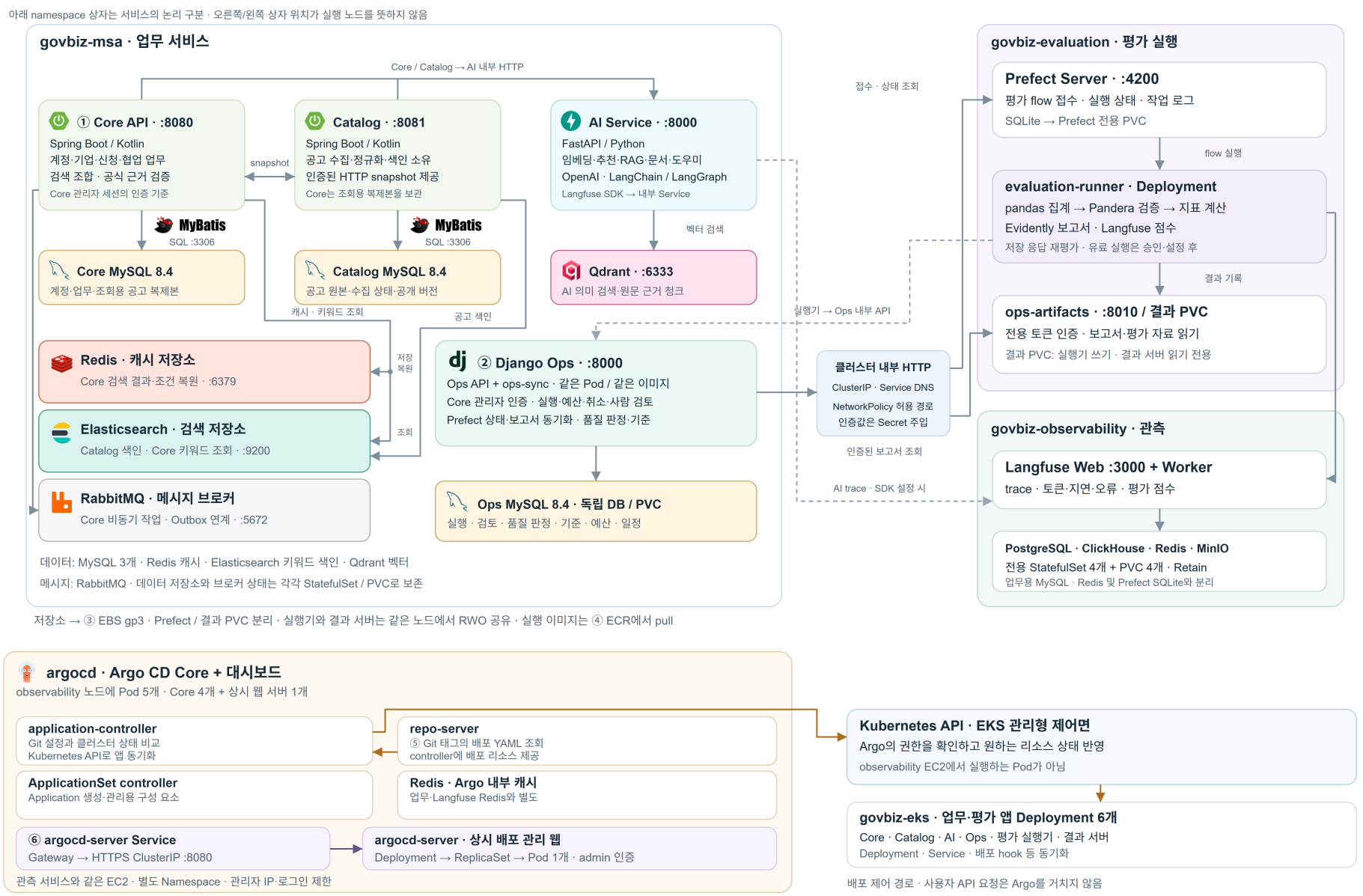
- **HPA · Pod 개수 조절** — 컨테이너 CPU requests 대비 평균 사용률 60% · Core·AI 각각 2~3개 · Gateway 1~3개
- **배치 대기 Pod** — 기존 노드의 자원·분산 조건으로 새 Pod를 배치할 수 없을 때
- **Cluster Autoscaler** — 대기 Pod를 배치할 노드 용량 확보 · application EC2 1대 추가
- **EC2 기본 2대 → 최대 3대** — application 1~2대 · 관측 노드 1대 · 새 노드가 준비되면 대기 Pod 실행

Core·AI는 같은 서비스끼리 EC2당 최대 1개 · 세 번째 Core 또는 AI Pod에는 세 번째 EC2가 필요

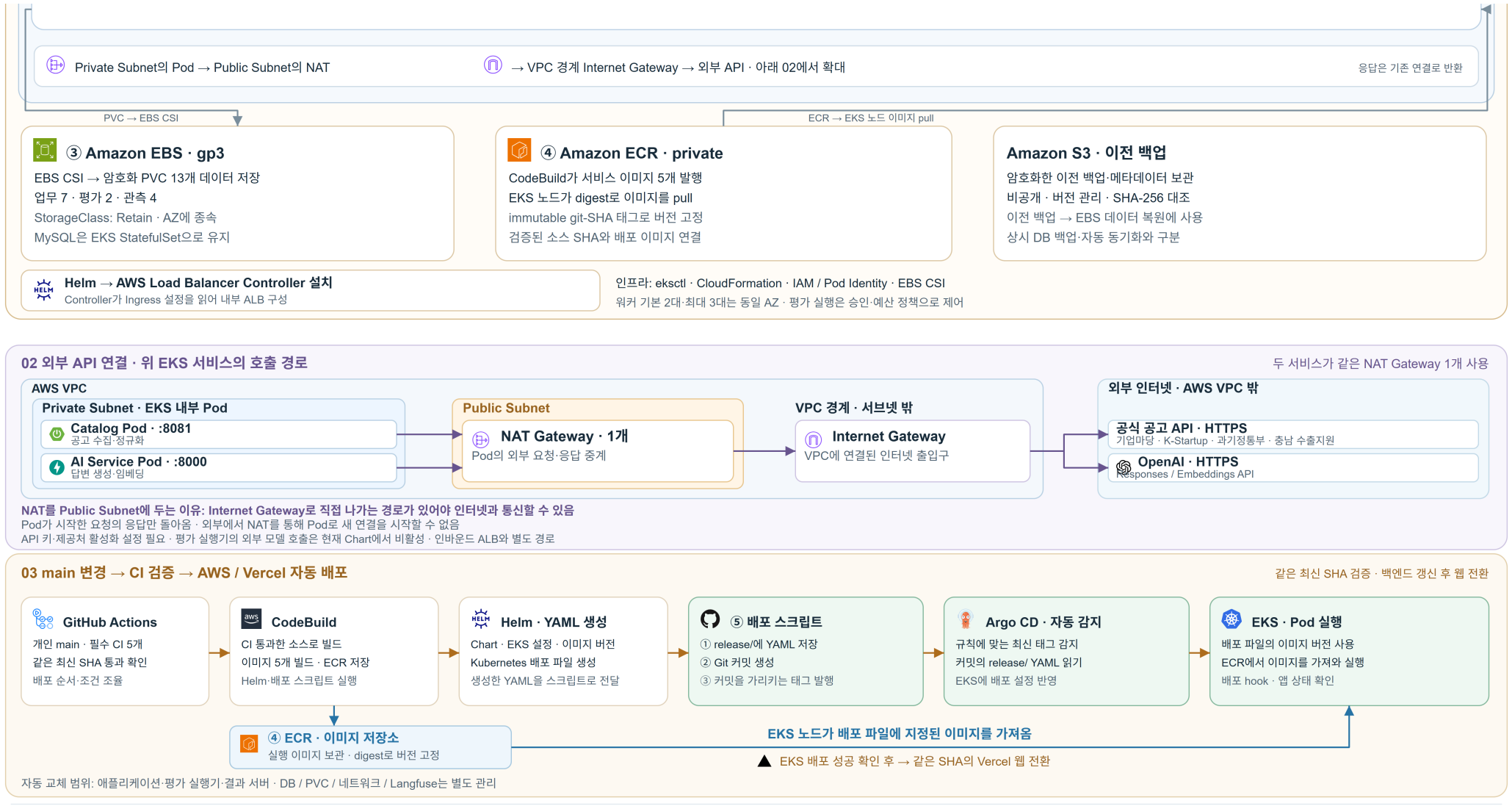
축소: HPA 5분 안정화 · 추가 노드가 10분간 불필요하고 남은 Pod를 안전하게 옮길 수 있어야 EC2 제거

아래 namespace 상자는 서비스의 논리 구분 · 오른쪽/왼쪽 상자 위치가 실행 노드를 뜻하지 않음

운영 구성도 ② 업무·평가·관측·Argo CD



운영 구성도 ③ 저장소·외부 API·자동 배포



현재 운영 설정 · 실제 배포·축소 관찰 기록은 구성도 설명 문서 참고 · 실행 완료 ≠ 품질 합격

Frontend: 클린 아키텍처 적용

공고 조회 등 주요 업무에서 Presentation과 Data는 Domain 계약을 참조하고, App이 구현체를 연결합니다.

Presentation

View·ViewModel Hook·화면 상태
사용자 입력과 결과 렌더링 담당
UseCase를 통해 업무 기능 실행

Domain

Entity·UseCase·Repository interface
업무 데이터와 데이터 접근 계약 정의
UseCase는 HTTP 구현을 직접 참조하지 않음

Data

RepositoryImpl·API·DTO
HTTP 응답 검증과 Domain 모델 변환
Domain의 Repository interface를 구현

App / DI

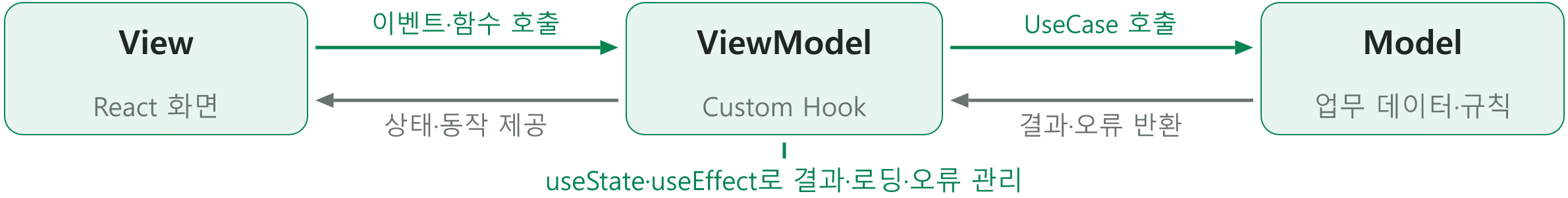
Awilix factory로 협력 객체 연결
UseCase 생성자에 Repository 구현체 전달
Hook은 앱 컨테이너에서 UseCase 조회

공고 상세 조회: ViewModel이 UseCase를 실행하고, 주입된 Repository 구현체가 HTTP API를 호출합니다.

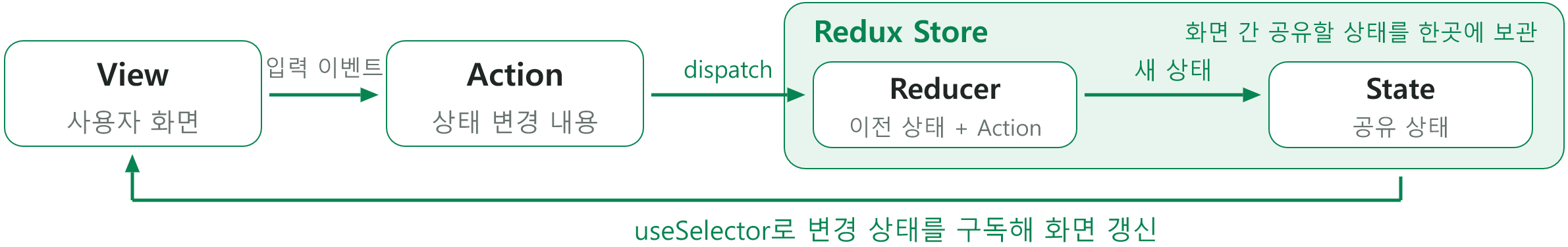
MVVM 패턴과 Flux 패턴의 구조

화면에 종속된 상태는 Custom Hook으로, 화면 간 공유할 상태는 Redux Store로 관리합니다.

Custom Hook 기반 MVVM 패턴



Redux Toolkit 기반 Flux 패턴



Backend: 레이어드 아키텍처

기능별 폴더 안에서 HTTP 처리·업무 흐름·DB 접근·외부 통신의 책임을 분리합니다.

Controller

HTTP 입력 검증·공개 Request/Response 변환
사용자 기능을 담당하는 Service 호출
DB Mapper와 외부 Client는 직접 호출하지 않음

Service · Domain

Service: 업무 실행 순서와 처리 조합
Domain: 업무 모델과 프레임워크 독립 규칙
Spring 생성자 DI로 필요한 협력 객체 연결

Repository · MyBatis

Repository: Domain·DbRow 및 JSON 변환
Mapper·XML로 SQL을 분리해 MySQL에 접근
DbRow는 Repository 밖으로 노출하지 않음

Facade · Client

Client: 외부 HTTP 통신·DTO·예외 처리
Facade: 복합 호출의 검증·변환을 묶을 때 사용
단순 외부 호출은 Service가 Client를 직접 사용

상세 DB 조회: Controller → Service → Repository → MyBatis Mapper → Mapper XML → MySQL

비동기 작업: RabbitMQ로 접수·실행 분리

요청을 먼저 저장하고, Core 내부 소비자가 큐에서 작업을 받아 실행합니다.

RabbitMQ 업무 경로

MySQL Outbox에 작업 저장
작업 ID만 RabbitMQ로 전달
Core 내부 소비자가 실행
결과 저장 후 ACK

중복 메시지는 DB 선점으로 방어
결과 불명은 UNKNOWN으로 기록
무조건 자동 재호출하지 않음

정기 리포트 생성·발송
AI 생성과 SMTP 발송은 서로 다른 큐
외부 응답 대기를 예약 스레드에서 분리

중복 지원 수혜 검토
요청 당시 입력을 고정하고 근거를 분석
202 접수 후 상태·저장된 결과 조회

공식 신청 문서 분석
사용자 요청의 첨부 수집·문항 추출은 큐로 실행
공고별 사전 분석은 별도 DB Outbox·Worker

카카오 탈퇴 연결 해제
탈퇴와 작업 저장을 같은 트랜잭션으로 처리
연결 해제 확인 전 같은 identity 재가입 차단

Redis: 로그인해도 검색 결과를 이어서

로그인 전 검색 결과와 조건을 30분 보관하고, 로그인 뒤 모델 재호출 없이 복원합니다.

01 비회원 검색

결과·조건 스냅샷 저장

최종 추천이 3~5건일 때 저장
화면에는 2건과 임시 결과 토큰
0~2건은 복원용 Redis를 쓰지 않음

02 로그인·소유권 확인

Lua로 원자적 계정 연결

회원 세션을 별도로 확인
최초 복원 계정에 소유권 부여
다른 계정의 복원은 거부

03 전체 결과 복원

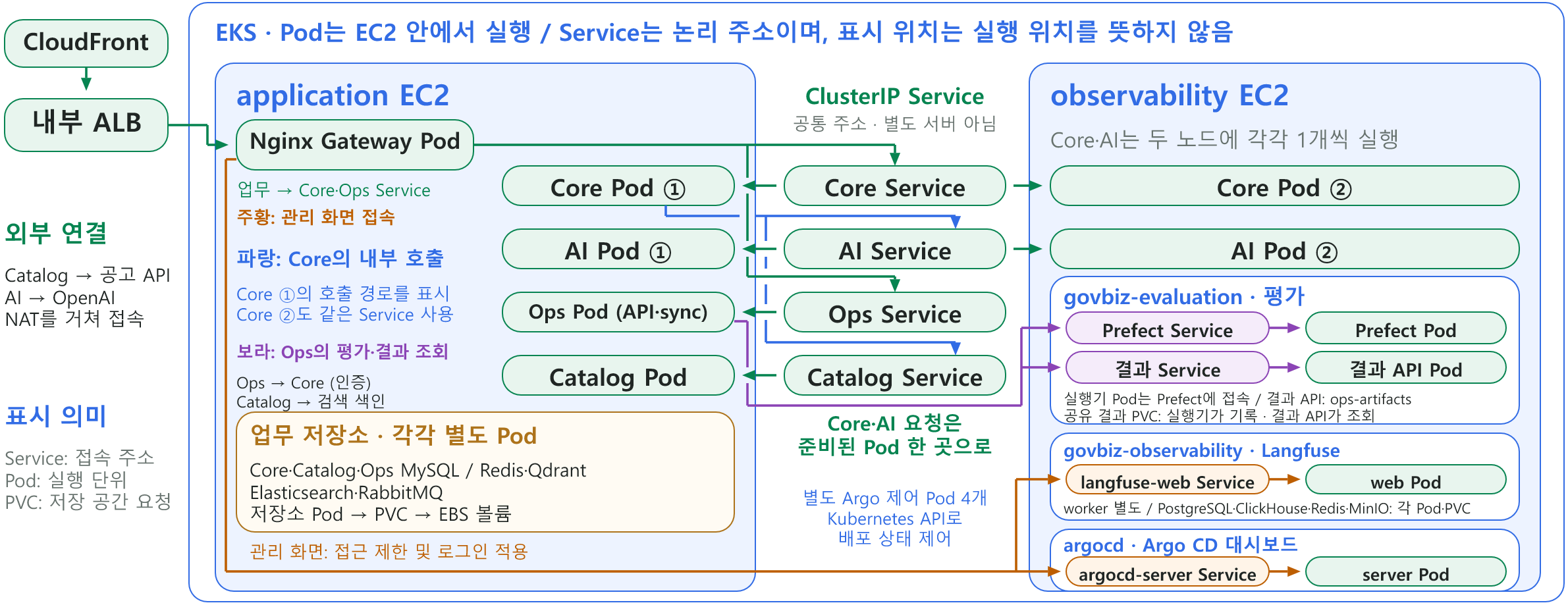
모델 재호출 없이 조회

전체 추천과 검색 조건 복원
조회해도 30분 TTL은 고정
공고·접수 상태·점수는 검색 당시 기준

resultToken은 보관된 검색 결과를 찾는 임시 번호이며, 복원에는 유효한 회원 세션이 필요합니다.

GovBiz의 Kubernetes 적용 아키텍처

Gateway 1개가 application EC2에서 실행되는 배치 예시입니다. Service는 Pod를 찾는 접속 주소입니다.



Core·AI·Catalog·Ops: govbiz-msa / Namespace는 논리 구분이며 EC2와 일대일 관계가 아닙니다.

Ops API와 MySQL: 요청 전달

Ops Service는 준비된 API Pod로 연결하고, Ops API는 MySQL Pod에 직접 접속합니다.

실제 요청 경로

Nginx Pod
로컬 web / EKS Gateway

→

Ops Service
ops-service · ClusterIP :8000

→

Ops Pod
Ops API 컨테이너 · HTTP :8000

→

MySQL Pod
ops-mysql-0 · MySQL :3306

Ops API 접속: 일반 ClusterIP Service

Pod가 교체되면 IP가 달라질 수 있습니다.
호출자는 일정한 Service 주소를 사용합니다.
Service는 라벨이 맞고 Ready 상태인 Pod로 연결합니다.

EndpointSlice에 대상 Pod의 주소와 준비 상태를 기록합니다.

MySQL 접속: Headless Service

ops-mysql은 clusterIP: None으로 설정합니다.
CoreDNS가 MySQL Pod의 IP를 알려줍니다.
Ops API는 해당 IP의 3306 포트로 직접 접속합니다.

Headless Service에는 별도 접속용 가상 IP가 없습니다.

초록 화살표가 요청 경로입니다. Deployment와 ReplicaSet은 API 요청이 지나가는 경로가 아닙니다.

Ops MySQL: PVC·PV와 데이터 보존

PVC는 사용할 저장 공간을 정하고, PV는 실제 EBS 디스크의 연결 정보를 관리합니다.

저장소 연결 관계

StatefulSet(ops-mysql)이 MySQL Pod를 생성·유지

MySQL Pod
DB 프로그램 실행
ops-mysql-0
PVC 이름을 지정해 사용

→ 지정

PVC
저장 공간 요청·할당
예: MySQL용 10Gi 요청
할당받은 PV와 연결

→ 연결

PV
디스크 연결 정보
EBS 볼륨 ID·용량 등
Kubernetes에 등록

→ 참조

AWS EBS
실제 디스크
MySQL 데이터 파일 저장
/var/lib/mysql에 마운트

PVC·PV는 Kubernetes 설정 정보이며, 실제 데이터는 EBS 디스크 한 곳에 저장됩니다.

위 화살표는 설정의 연결 관계입니다. MySQL은 마운트된 EBS에 DB 파일을 읽고 씁니다.

Pod가 교체되면

새 MySQL Pod 실행 → **기존 PVC·PV에 다시 연결** → **같은 EBS의 데이터 사용**

기존 PVC·PV·EBS를 유지할 때 데이터가 이어집니다. 삭제한 저장소의 자동 복구를 뜻하지 않습니다.

Pod·EC2 자동확장과 분산 배치

Pod 수를 먼저 조절하고, 자원·분산 조건을 만족할 노드가 부족하면 EC2를 추가합니다.

01 CPU 사용량에 따른 Pod 수 조절

Metrics Server
컨테이너 CPU 사용량 수집

→

Horizontal Pod Autoscaler
Deployment가 관리하는 Pod 개수 조절

→

Deployment
ReplicaSet이 필요한 Pod 생성

기준: 컨테이너 CPU requests 대비 평균 60% / Core·AI 각각 2~3개, Gateway 1~3개

Horizontal Pod Autoscaler는 EC2 전체 CPU나 요청 수를 직접 보지 않으며, Pod 축소에는 5분 안정화 구간을 둡니다.

02 배치할 수 없는 Pod가 생기면 EC2 추가

대기 중인 Pod
자원 부족 또는 노드 분산 조건

→

Cluster Autoscaler
application EC2를 1대에서 2대로

→

EC2 준비 · Pod 배치
Scheduler가 새 노드에 배치

observability 1대는 유지합니다. EC2는 기본 2대·최대 3대이며, Core·AI는 같은 서비스끼리 노드를 분리합니다.
세 번째 Core·AI Pod에는 세 번째 EC2가 필요합니다. Horizontal Pod Autoscaler·Autoscaler는 요청을 중계하지 않습니다.
추가 EC2는 10분간 불필요하고 Pod를 안전하게 옮길 수 있을 때 축소합니다. PDB의 퇴거 제한도 확인합니다.

Qdrant 저장: 공고와 근거 청크를 분리

검색 목적에 따라 저장 단위와 컬렉션을 구분합니다.

01 AI 대화 검색 · 공고 단위

공식 API 공고
제공처별 수집·정규화

→

검색 텍스트 구성
제목·기관·지원 대상·지역·분야·신청기간·요약
K-Startup: 업력·대상·대표자 연령 분류(있는 경우)
(분류 정보는 검색용이며 신청 자격의 근거는 아님)

→

Embedding
OpenAI 임베딩
공고 검색 문서 1건당
벡터 1개

→

Qdrant
공고용 컬렉션에 저장

저장 내용 벡터 + 공고 ID + 내용 해시 + 제공처 코드

02 공고 원문 RAG · 청크 단위

활용: 상세 공고 질문 · GovBiz 도우미의 관심 공고 질문 · 관리자 Gov 에이전트의 원문 질문

공식 HTML
기업마당·K-Startup
상세 본문 (첨부파일 제외)

→

전처리
본문 추출·태그 제거
공백·텍스트 정리

→

Chunking
청크당 최대 1,500자
문서당 50개 (초과 시 오류)
문자 수는 UTF-16 기준

→

Embedding
OpenAI로 청크별
벡터 생성

→

Qdrant
청크 1개당
벡터 1개 저장

저장 내용 벡터 + 청크 원문 + 문서 ID + 청크 ID·내용 해시·순서

공고 데이터·정제 원문은 MySQL에 보존하며, 두 경로는 별도 Qdrant 컬렉션을 사용합니다.
같은 내용·임베딩 설정의 문서·청크 벡터는 재사용하고, 누락된 벡터만 생성합니다.

Elasticsearch 저장과 하이브리드 검색

수집한 공고는 메모리에 보관한 뒤 검색 자료를 만들고, 두 준비가 성공하면 Catalog MySQL에 저장합니다.

새 공고 A를 가져왔을 때의 저장 순서

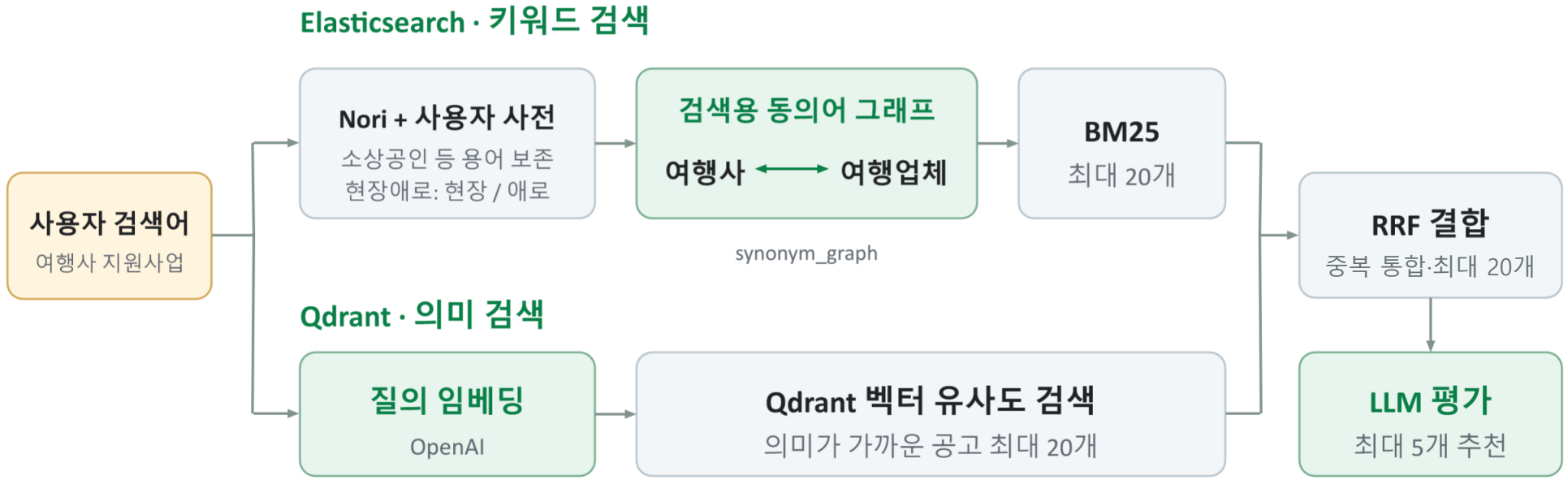
1. 공식 API에서 공고 수집
Catalog가 가져온 공고를
프로그램 메모리에 보관
제목·지원 대상·신청 기간 등

→

2. 검색에 쓸 자료 저장
Elasticsearch: 공고 문서·키워드 색인
Qdrant: 임베딩 벡터 저장
두 저장소의 준비 성공 확인

→

3. 둘 다 성공하면 MySQL 저장
Catalog MySQL: 신규 추가·내용 갱신
Core가 최신 목록을 가져와
Core MySQL에도 반영



검색 준비가 실패하면 기존 공고 목록을 유지합니다. 동의어 확장은 저장이 아닌 검색어 분석에 적용합니다.

실제 검색은 키워드 → 의미 검색 → 후보 결합 순서입니다. 관련도와 자격을 분리하고 자격 미확인은 감점하지 않습니다.

검색 방식별 성능 비교

기업마당 공고 1,422건 · 질문 16개 평가 · 관련 공고가 있는 12문항 평균

AI 관련 판정 47쌍 기준	Recall@20	Recall@5	MRR@5
기존 의미검색 기관·신청 기간 포함	0.5243	0.4410	0.6278
개선 의미검색 제목·대상·내용·분야·지역	0.5139	0.4410	0.6875
기존 RRF 기존 본문 · k=60	0.7479	0.4764	0.7222
개선 RRF 개선 본문 · k=1 · 동점 BM25 우선	0.7479	0.4764	0.8542
BM25 단독 Elasticsearch · 기존 본문 유지	0.7417	0.5410	0.8403

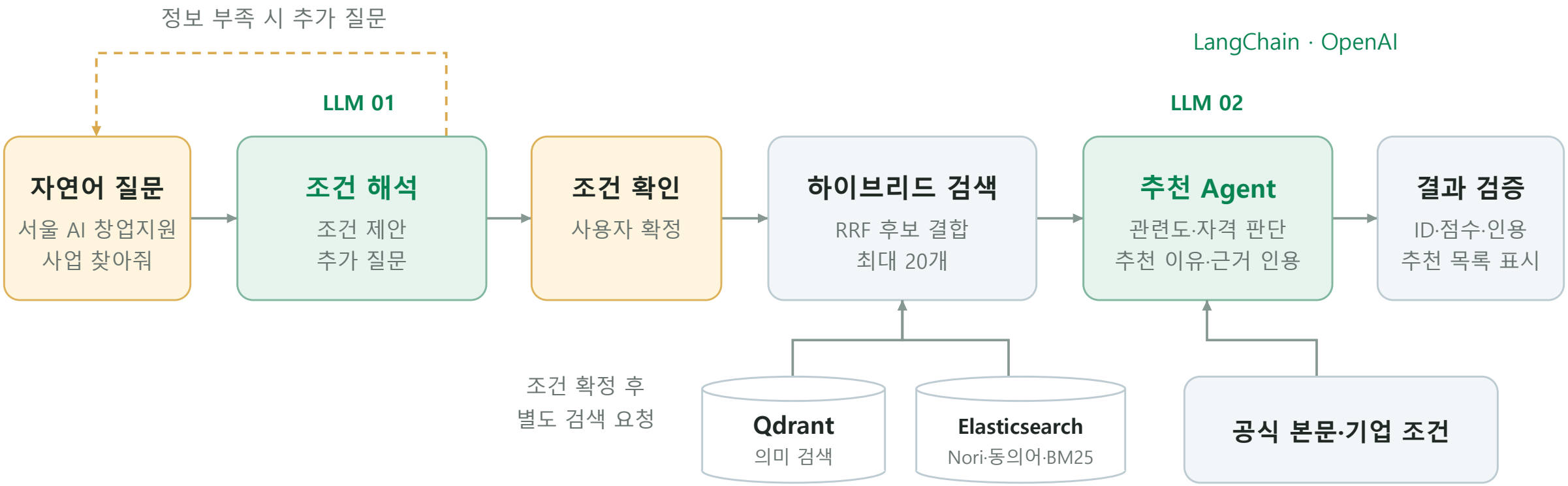
RRF 개선	Recall@20·Recall@5 유지 · MRR@5 상승 · 일부 질문은 순위 하락
비교 해석	BM25는 Recall@5 우세 · 의미검색 단독 Recall@20은 소폭 하락

AI 참조 878쌍 중 관련 47·미확정 30쌍 · 관련 판정 없는 4문항은 평균 제외 · 풀 밖 미검토 공고 미포함

같은 평가셋을 본 뒤 조정한 결과 · 사람 검수·독립 검증 아님 · 최종 추천 LLM 미측정 · 2026-10-15

AI 대화 검색 에이전트 흐름

사용자가 조건을 확인한 뒤 검색하고, 검색 근거로 추천을 생성합니다.



신청 문서 작성 에이전트 흐름

Agent의 배치 계획을 파일 형식에 맞춰 MCP 도구와 Core 편집기가 실행합니다.

01 양식 구조 분석

공식 첨부파일
HWP · HWPX · PDF

↓

Core 블록·위치 추출

↓

양식 분석 Agent
문항 발견·입력칸 매핑

kordoc MCP · 필요 시 읽기 보조

02 답변 입력과 저장

사용자 답변
문항별 직접 입력·수정

↓

답변 자동 저장
입력값 저장·버전 확인

↓

초안 파일 생성 요청
저장된 답변으로 파일 생성

03 원본 기입과 검증

배치 계획 Agent
저장된 값·원본 위치 연결

↓

MCP 도구 실행

HWPX · Hangeul-mcp
구조 확인·미리보기·기입·검증

PDF · pdf-edit-mcp
읽기·예시 삭제 / Core PDFBox 기입

↓

HWP · Core hwplib 직접 실행

↓

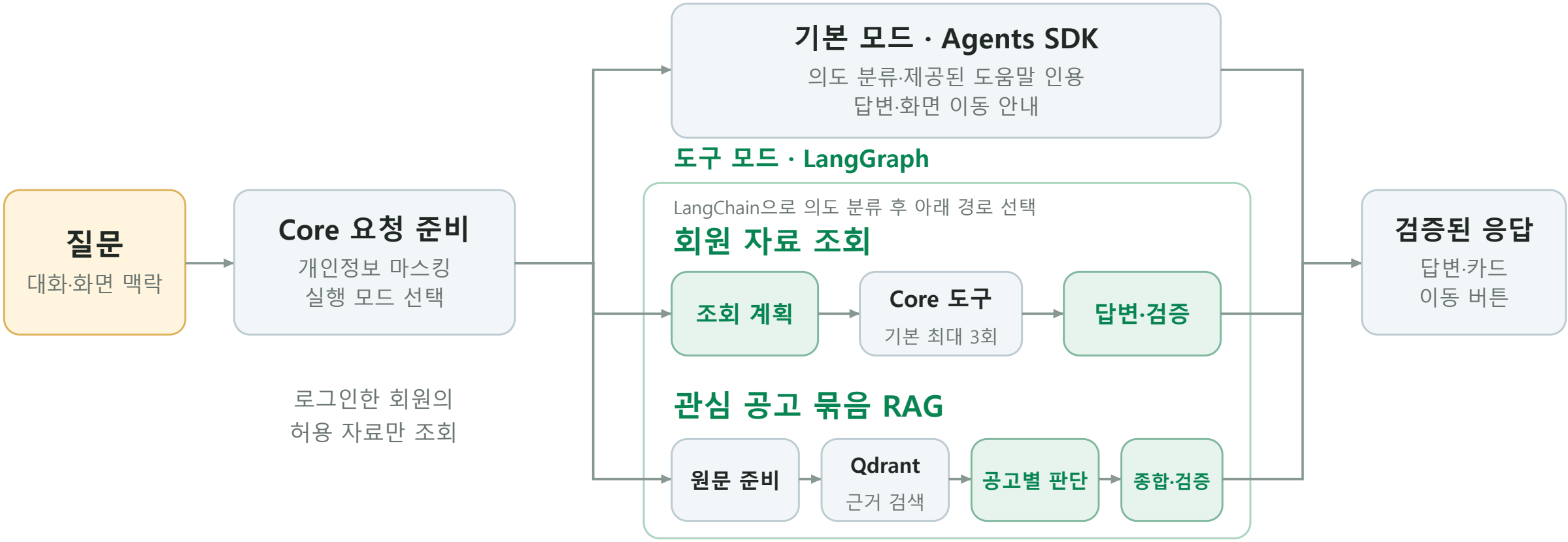
재검증·사용자 검토·다운로드

답변 해석·문항 초안 API는 별도로 지원하며, 현재 웹은 직접 입력·자동 저장을 사용합니다.

LLM: ApplicationPreparationAgent · LangChain · OpenAI

GovBiz 도우미의 의도별 실행 흐름

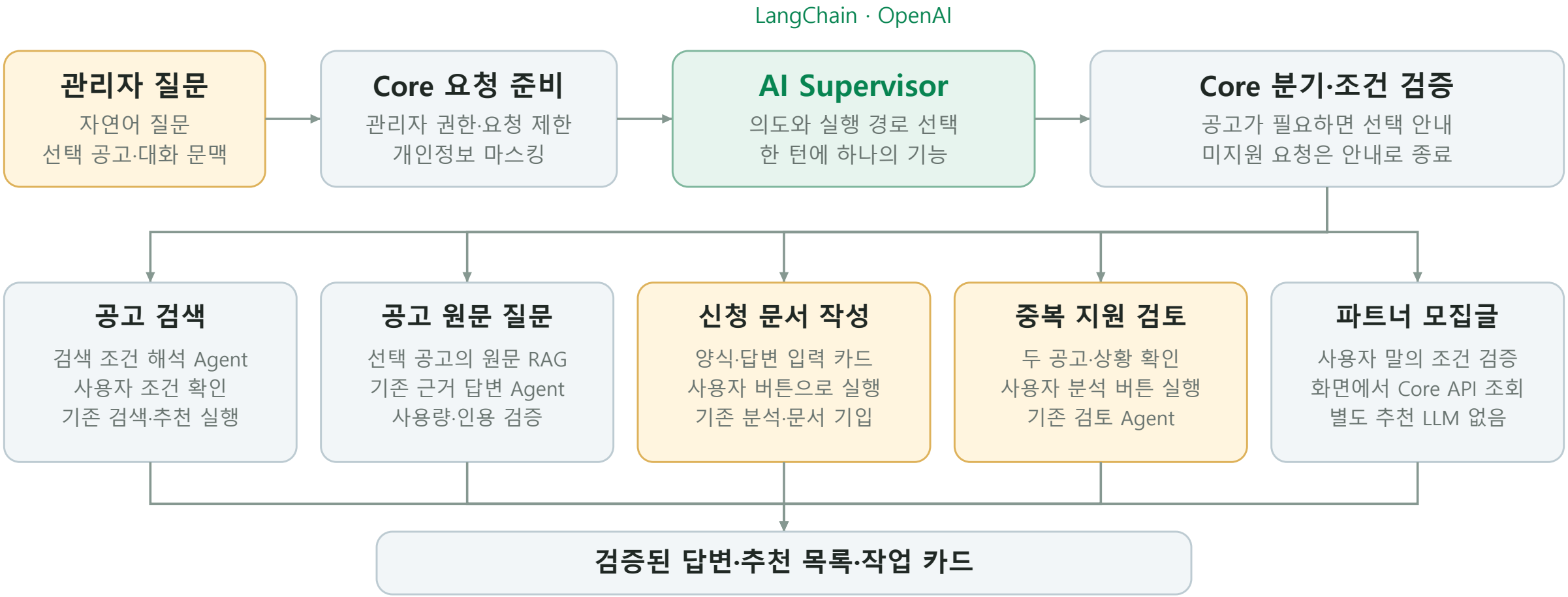
Core가 기본 모드와 도구 모드를 선택하고, 각 모드에서 의도 분류와 응답 검증을 수행합니다.



기본 모드는 Core가 답변과 이동 경로를 구성하며, 실제 검색·신청은 해당 기능 화면에서 진행합니다.

Gov 에이전트의 Supervisor 실행 흐름

관리자 요청의 의도를 판단하고, Core가 조건을 검증해 기존 에이전트와 업무 기능으로 연결합니다.



웹 작업 버튼은 해당 기능을 직접 실행합니다. Supervisor가 여러 작업을 연속 실행하거나 기관에 제출하지 않습니다.